

GCU Events Management System - JWT REST API Expansion

Owen Lindsey

Professor Sluiter, Shad

CST-407

Grand Canyon University

Project Overview

This milestone extends the Spring Boot security foundation by delivering a full-featured REST API secured with JSON Web Tokens (JWT). Two controller layers were introduced: `EventsApiController` delivers CRUD endpoints for the `eventsapp` schema, while `UsersApiController` manages registration, authentication, and token inspection. With JWT enforced at the filter chain, the application now supports stateless integration scenarios alongside the existing Thymeleaf experience showcased in Milestone 1.

Key goals:

- REST endpoints hosted at `/api/events` and `/api/users`, organized with OpenAPI metadata for quick discovery.
- Secure create/update/delete/event-admin flows accessible only to JWT-authenticated users.
- Public read/search endpoints enabling anonymous discovery of event content.
- Documentation-first workflow through Swagger UI (and Postman parity) to accelerate partner onboarding.

Video Demonstration

Link to 5-minute demonstration video: <https://youtu.be/qoBsE9jP9jU>

The Milestone 2 video will highlight:

1. **JWT Login Flow** – obtaining a token via `/api/users/login` and storing it in Swagger's or Postman's Authorization context.
2. **Secure CRUD** – creating, updating, and deleting events with the `Authorization: Bearer <token>` header, contrasted against rejected unauthenticated attempts.
3. **Public Access** – browsing `/api/events` and `/api/events/search` without credentials.
4. **User Lifecycle** – registering a new account, logging in, and verifying `/api/users/me`.

Screenshots captured during rehearsal are archived in the `@Photos` album for quick reference; the final recording will mirror those flows.

API Architecture

Events API (/api/events)

- GET /api/events – Lists all events; open to everyone.

GET/api/events

Parameters

Cancel

No parameters

ExecuteClear

Responses

Curl

```
curl -X 'GET' \
'http://localhost:8080/api/events' \
-H 'accept: */*'

```

Request URL

```
http://localhost:8080/api/events

```

Server response

CodeDetails

200

Response body

```
[
  {
    "id": "2",
    "name": "Grand Opening",
    "date": "2025-10-08",
    "location": "Phoenix, AZ, USA",
    "organizerid": "1",
    "description": "free food! Bring a friend."
  },
  {
    "id": "3",
    "name": "Birthday",
    "date": "2024-08-08",
    "location": "Scottsdale, AZ, USA",
    "organizerid": "1",
    "description": "Bring a gift"
  }
]

```

Response headers

```
cache-control: no-cache,no-store,max-age=0,must-revalidate
connection: keep-alive
content-type: application/json
date: Thu,30 Oct 2025 22:02:17 GMT
expires: 0
keep-alive: timeout=60
pragma: no-cache
transfer-encoding: chunked
vary: Origin,Access-Control-Request-Method,Access-Control-Request-Headers
x-content-type-options: nosniff
x-frame-options: DENY
x-xss-protection: 0

```

Responses

CodeDescriptionLinks

200OK

No links

Media type

/

Controls Accept header.

Example Value | Schema

```
[
  {
    "id": "string",
    "name": "string",
    "date": "2025-10-30T22:02:17.512Z",
    "location": "string",
    "organizerid": "string",
    "description": "string"
  }
]

```

- GET /api/events/{id} – Retrieves a single record or returns HTTP 404.

GET

/api/events/{id}

Parameters

Cancel

Name	Description
id ★ required	
string	1
(path)	

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:8080/api/events/1' \
  -H 'accept: */*'
```

Request URL

```
http://localhost:8080/api/events/1
```

Server response

Code	Details
404	Error: response status is 404

Undocumented

Response headers

```
cache-control: no-cache,no-store,max-age=0,must-revalidate
connection: keep-alive
content-length: 0
date: Thu,30 Oct 2025 21:59:42 GMT
expires: 0
keep-alive: timeout=60
pragma: no-cache
vary: Origin,Access-Control-Request-Method,Access-Control-Request-Headers
x-content-type-options: nosniff
x-frame-options: DENY
x-xss-protection: 0
```

Responses

Code	Description	Links
200	OK	No links

Media type

/

Controls Accept header.

Example Value | Schema

```
{
  "id": "string",
  "name": "string",
  "date": "2025-10-30T21:59:42.195Z",
  "location": "string",
  "organizerid": "string",
  "description": "string"
}
```

- GET /api/events/search?q=term – Full text search across event descriptions.

GET/api/events/search

Parameters

Cancel

Name	Description
q ★ required string (query)	free

Execute

Clear

Responses

Curl

curl -X 'GET' \

'http://localhost:8080/api/events/search?q=free' \

-H 'accept: */*'

Request URL

http://localhost:8080/api/events/search?q=free

Server response

Code	Details
200	Response body [{ "id": "2", "name": "Grand Opening", "date": "2025-10-08", "location": "Phoenix, AZ, USA", "organizerid": "1", "description": "free food! Bring a friend." }] Download Response headers cache-control: no-cache,no-store,max-age=0,must-revalidate connection: keep-alive content-type: application/json date: Thu,30 Oct 2025 22:04:29 GMT expires: 0 keep-alive: timeout=60 pragma: no-cache transfer-encoding: chunked vary: Origin,Access-Control-Request-Method,Access-Control-Request-Headers x-content-type-options: nosniff x-frame-options: DENY x-xss-protection: 0

Responses

Code	Description	Links
200	OK Media type */* Controls Accept header. Example Value Schema [{ "id": "string", "name": "string", "date": "2025-10-30T22:04:29.368Z", "location": "string", "organizerid": "string", "description": "string" }]	No links

- POST /api/events – Creates a new event; JWT required.

- `PUT /api/events/{id}` – Updates an existing event; JWT required.

PUT

/api/events/{id}

Parameters

Cancel

Reset

Name	Description
id * required	
string	2
(path)	

Request body required

application/json

```
{  
  "id": "string",  
  "name": "New Things",  
  "date": "2025-10-30T22:00:12.092Z",  
  "location": "string",  
  "organizerid": "ME",  
  "description": "This is for fun"  
}
```

Execute

Clear

Responses

Curl

```
curl -X 'PUT' \  
  'http://localhost:8080/api/events/2' \  
  -H 'accept: */*' \  
  -H 'Authorization: Bearer yJhbGciOiJIUzUxMiJ9.eyJyY2xlcjI6WyJST0xFMTVTRVIiXSwic3ViIjoib21vaSI6ImV2ZW50c2FwcCI6ImldCI6MTc2MTCg'  
  -H 'Content-Type: application/json' \  
  -d '{  
    "id": "string",  
    "name": "New Things",  
    "date": "2025-10-30T22:00:12.092Z",  
    "location": "string",  
    "organizerid": "ME",  
    "description": "This is for fun"  
  }'
```

Request URL

http://localhost:8080/api/events/2

Server response

Code

Details

- `DELETE /api/events/{id}` – Removes an event; JWT required.

DELETE

/api/events/{id}

Parameters

Cancel

Name	Description
id * required string (path)	2

Execute

Clear

Responses

Curl

```
curl -X 'DELETE' \
'http://localhost:8080/api/events/2' \
-H 'accept: */*' \
-H 'Authorization: Bearer yJhbGciOiJIUzUxMiJ9.eyJyb2xlcyI6WyJST0xFX1VTRVIiXSwic3ViIjoib21vaSIsImZlcyI6ImV2ZW50c2FwcCI6ImldCI6MTc2MTg'
```

Request URL

```
http://localhost:8080/api/events/2
```

Server response

Code

Details

Validation is applied through `@Valid` and `EventModel`, keeping the API consistent with the service layer. Responses return `EventModel` payloads to align with future mobile or SPA consumers.

Users API (`/api/users`)

- `POST /api/users/register` – Accepts `RegisterRequest` to create a user backed by BCrypt hashing.

Users

User registration, login, and token inspection

POST

/api/users/register

Parameters

No parameters

Cancel

Reset

Request body

required

application/json

```
{
  "userName": "omni",
  "password": "test"
}
```

Execute

Clear

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8080/api/users/register' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "userName": "omni",
    "password": "test"
  }'
```

Request URL

```
http://localhost:8080/api/users/register
```

Server response

Code	Details
201	Response body <pre>{ "id": "2", "userName": "omni", "roles": ["ROLE_USER"] }</pre> Download Response headers <pre>cache-control: no-cache,no-store,max-age=0,must-revalidate connection: keep-alive content-type: application/json date: Thu, 58 Oct 2025 21:55:28 GMT expires: 0 keep-alive: timeout=60 pragma: no-cache transfer-encoding: chunked vary: Origin,Access-Control-Request-Method,Access-Control-Request-Headers x-content-type-options: nosniff x-frame-options: DENY x-xss-protection: 0</pre>

Responses

Code	Description	Links
200	OK	No links

Media type

/

Controls Accept header

Example Value | Schema

```
{}
```

- `POST /api/users/login` – Authenticates credentials and returns `AuthResponse` (JWT, expiration, user payload).

POST

/api/users/login

Parameters

No parameters

Request body required

application/json

{
 "user": "admin",
 "password": "test"
}

ExecuteClear

Responses

Curl

```
curl -X 'POST' \  
  'http://localhost:8080/api/users/login' \  
  -H 'accept: */*' \  
  -H 'Content-Type: application/json' \  
  -d '{  
    "user": "admin",  
    "password": "test"  
  }'
```

Request URL
http://localhost:8080/api/users/login

Server response

Code	Details
200	Response body { "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXLTQwMjYxOTVlbnVudDQyZmFscCIsImtldCI6MTc2NTQ1ODQ0NDQ0LmVudDQyZmFscCIsImNpdCI6MTc2NTQ1ODQ0NDQ0LmVudDQyZmFscCJ9", "tokenType": "Bearer", "expiresIn": "3600000", "user": { "id": "2", "user": "admin", "roles": ["ROLE_USER"] } } Download

Response headers

cache-control: no-cache,no-store,max-age=0,must-revalidate
connection: keep-alive
content-type: application/json
date: Thu, 08 Oct 2025 21:57:55 GMT
expires: 0
keep-alive: timeout=60
pragma: no-cache
transfer-encoding: chunked
vary: Origin,Access-Control-Request-Method,Access-Control-Request-Headers
x-content-type-options: nosniff
x-frame-options: DENY
x-xss-protection: 0

Responses

Code	Description	Links
200	OK	No links

Media type

/

Control Accept header

Example Value | Schema

{}

- GET /api/users/me – Returns the authenticated profile; secured by @PreAuthorize("isAuthenticated()").

GET

/api/users/me

Parameters

No parameters

Execute

Clear

Responses

Curl

curl -X 'GET' \n'http://localhost:8080/api/users/me' \n-H 'accept: */*'

Request URL

http://localhost:8080/api/users/me

Server response

Code

Details

200

Response body

<!-- login.html -->\n<!DOCTYPE html>\n<!-- layout.html -->\n<html>\n<head>\n\n<title>Login - GCU Events App</title>\n<link rel="stylesheet" href="/css/styles.css">\n<meta name="viewport" content="width=device-width, initial-scale=1.0">\n\n</head>\n<body>\n\n<!-- GCU Header -->\n<div class="gcu-header">\n GRAND CANYON UNIVERSITY
\n Events Management System\n</div>\n\n<!-- Navigation -->\n<nav>\n \n Home\n\n <!-- Show register and login links only when user is not authenticated -->\n \n

Response headers

cache-control: no-cache,no-store,max-age=0,must-revalidate\nconnection: keep-alive\ncontent-language: en-US\ncontent-type: text/html; charset=UTF-8\ndate: Thu, 30 Oct 2025 21:58:31 GMT\nexpires: 0\nkeep-alive: timeout=60\npragma: no-cache\ntransfer-encoding: chunked\nvary: Origin,Access-Control-Request-Method,Access-Control-Request-Headers\nx-content-type-options: nosniff\nx-frame-options: DENY\nx-xss-protection: 0

Responses

Code

Description

Links

200

OK

No links

Media type

/

Control: Accept header.

Example Value | Schema

{}

Role assignment defaults to `ROLE_USER`; additional roles can be added without modifying controller logic because Spring Security inspects the JWT claims exposed by `JwtService`.

JWT Authentication Flow

- Login** – `/api/users/login` authenticates against `AuthenticationManager` using `UsernamePasswordAuthenticationToken`.
- Token Issuance** – `JwtService.generateToken` issues a signed token with issuer, expiration, and username claims.
- Request Filtering** – `JwtAuthenticationFilter` intercepts `/api/**` traffic, validating tokens and populating the `SecurityContext`.
- Authorization** – `SecurityConfig` gates sensitive routes; unauthenticated calls receive HTTP 401 via `JwtAuthenticationEntryPoint`.
- Stateless Sessions** – `SessionCreationPolicy.STATELESS` ensures scalability for API clients and microservice consumers.

The secret, issuer, and expiration settings live in `application.properties`. Updates to those values invalidate existing tokens, reinforcing controlled access.

API Tooling & Demonstration Notes

Swagger UI

- Hosted at `http://localhost:8080/swagger-ui/index.html`.
- Uses the OpenAPI configuration (`OpenApiConfig`) to expose the `bearerAuth` security scheme.
- “Authorize” button accepts the JWT from the login response. Once authorized, Swagger persists the header for subsequent secured endpoints.
- Screenshots in `@Photos` capture login, authorized POST/PUT/DELETE flows, and 401 responses when omitting the token.

Postman Parity (per assignment instructions)

- Login Collection** – Send `POST http://localhost:8080/api/users/login` with JSON body:

```
{
  "userName": "root",
  "password": "root"
}
```

- Set Environment Variable** – Store the `token` field as `{{jwt_token}}`.
- Protected Call** – Example update:

```
PUT http://localhost:8080/api/events/3
Authorization: Bearer {{jwt_token}}
Content-Type: application/json
```

- Negative Test** – Duplicate the request without Authorization header to capture the 401 response required by the rubric.

Database & Data Seeding

- MariaDB 12.0.2 running locally on Arch Linux (`systemctl enable --now mariadb`).
- `eventsapp.sql` loaded into `eventsapp` schema with seeded `users`, `roles`, and `events` tables.
- `AuthenticationManager` relies on `CustomUserDetailsService`, which fetches users from the seeded data (initial admin user: `root` / `root`).

Testing & Verification Checklist

- `GET /api/events` – succeeds without token.
- `POST /api/events` – succeeds with token, fails with 401 without token.
- `PUT /api/events/{id}` – similar success/failure scenarios recorded.
- `DELETE /api/events/{id}` – confirms audit trail by removing test data created earlier in the session.
- `GET /api/users/me` – returns authenticated user details when token is present.

Logs (`spring.jpa.show-sql=true` and `spring.jpa.hibernate.ddl-auto=update`) assist in verifying database persistence during the demo.